Лекція №5 (2 години). CSS Grid та методології адаптивного дизайну.

План лекції

1. Вступ: Різниця між одновимірним та двовимірним макетуванням (Flexbox vs Grid).
2. Основи CSS Grid: Контейнер, напрямні лінії, клітинки, смуги та області.
3. Властивості Grid-контейнера: створення сітки (`grid-template-columns` , `grid-template-rows`).
4. Автоматичне розміщення (`auto-fill` , `auto-fit` , `minmax()`).
5. Властивості Grid-елементів: розміщення по сітці (`grid-column` , `grid-row`).
6. Адаптивний дизайн: медіа-запити (Media Queries).
7. Методології адаптивного дизайну: Mobile First vs Desktop First.

Перелік умовних скорочень

- **CSS** (Cascading Style Sheets) — каскадні таблиці стилів.
- **RWD** (Responsive Web Design) — адаптивний веб-дизайн.
- **fr** (Fraction) — фракція, гнучка одиниця вимірювання в CSS Grid.
- **UI** (User Interface) — інтерфейс користувача.

Вступ

Flexbox зробив революцію у макетуванні, але він призначений переважно для вирішення одновимірних завдань — розміщення елементів в один рядок або в одну колонку. Коли ж постає необхідність розробити складну загальну розкладку сторінки (наприклад, визначити розташування заголовка, футера, бокових панелей і головного контенту), Flexbox стає недостатньо ефективним, змушуючи розробників вкладати багато контейнерів один в один.

Тут на допомогу приходить **CSS Grid Layout** (скорочено Grid) — найпотужніша система двовимірного компонування в CSS. Grid дозволяє будувати макети, маніпулюючи одночасно і рядками, і колонками. Це надає безпрецедентний контроль над розташуванням кожного блоку на екрані.

Крім того, сучасний веб вимагає, щоб сайти однаково добре виглядали як на великих моніторах, так і на маленьких екранах смартфонів. Створення таких інтерфейсів є завданням **Адаптивного веб-дизайну**. У цій лекції ми також розглянемо медіа-запити та сучасні стратегії адаптації контенту.

1. Flexbox vs Grid

Головна відмінність між системами компонування:

- **Flexbox є одновимірним.** Він розподіляє простір уздовж однієї осі (рядка АБО колонки).
- **Grid є двовимірним.** Він працює з рядками ТА колонками одночасно.

Практичне правило:

- Використовуйте **Flexbox** для вирівнювання елементів всередині конкретного компонента (навігаційне меню, група кнопок, вміст картки).
- Використовуйте **Grid** для побудови загального макета сторінки (структура секцій, сітка товарів).

У більшості сучасних проектів ці дві технології використовуються разом, ідеально доповнюючи одна одну.

2. Основи CSS Grid: Термінологія

Як і у Flexbox, Grid базується на взаємодії батьківського елемента (Grid Container) та дочірніх (Grid Items).

Ключові поняття:

- **Grid-лінії (Grid Lines):** Вертикальні та горизонтальні лінії, які формують сітку. Вони мають номери (від 1 і далі, або від'ємні значення з кінця).
- **Grid-смуга (Grid Track):** Простір між двома сусідніми grid-лініями (може бути як колонкою, так і рядком).
- **Grid-клітинка (Grid Cell):** Найменша одиниця на сітці, простір між двома сусідніми вертикальними та горизонтальними лініями (схоже на клітинку Excel).
- **Grid-область (Grid Area):** Будь-яка прямокутна зона, утворена з кількох клітинок. Область завжди повинна бути прямокутником (не можна створити зону у формі літери "Г").

3. Властивості Grid-контейнера

Щоб перетворити елемент на сітку, йому призначають:

```
.container {
  display: grid; /* Вмикає Grid */
}
```

Створення сітки

Головні властивості для описання структури:

- `grid-template-columns` — визначає кількість та ширину колонок.
- `grid-template-rows` — визначає кількість та висоту рядків.

Ключове нововведення — одиниця `fr` (Fraction): `fr` репрезентує долю(фракцію) доступного простору.

```
.grid {
  display: grid;
  /* 3 колонки: перша займає 200px, а весь залишок ділиться між
     наступними як 1 до 2 */
  grid-template-columns: 200px 1fr 2fr;
  /* 2 рядки по 100px кожен */
  grid-template-rows: 100px 100px;
  /* Відступи між клітинками (заміняє margin) */
  gap: 20px;
}
```

Функція `repeat()`

Дозволяє скоротити запис, якщо сітка має багато однакових колонок: `grid-template-columns: repeat(4, 1fr);` — створить 4 рівні колонки, кожна по `1fr`.

4. Автоматичне розміщення (`auto-fill` , `auto-fit` , `minmax()`)

Особлива магія Grid полягає в здатності створювати адаптивні сітки без використання медіа-запитів. Для цього використовується функція `minmax()`, що задає діапазон мінімального і максимального розміру (наприклад, `minmax(250px, 1fr)`).

У комбінації з `auto-fit` браузер автоматично перенесе елементи, якщо вони не "влазять" у рядок:

```
.gallery {
  display: grid;
  /* Розмісти скільки завгодно колонок у рядку.
     Мінімальна ширина кожної — 200px.
     Якщо у рядку залишиться вільне місце — розтягни колонки порівну (1fr). */
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

```
  gap: 15px;
}
```

Це найпопулярніший патерн для сітки карток з товарами.

5. Властивості Grid-елементів (розміщення на сітці)

Визначивши сітку на контейнері, ми можемо розташувати в ній елементи, використовуючи властивості: `grid-column-start`, `grid-column-end`, `grid-row-start`, `grid-row-end`.

Найчастіше використовують їх скорочення: `grid-column` та `grid-row`. Значення задаються у вигляді *лінія початку / лінія кінця*.

```
.item-a {
  /* Елемент почнеться від 1-ї вертикальної лінії і закінчиться на 3-й
     (буде займати 2 колонки) */
  grid-column: 1 / 3;

  /* Почнеться на 2-му рядку і розтягнеться до самого кінця (-1) */
  grid-row: 2 / -1;
}
```

Також можна вказати, скільки клітинок елемент має зайняти, використовуючи ключове слово `span`: `grid-column: span 2;` — об'єднати дві клітинки по горизонталі.

Метод іменованих областей (Grid-Template-Areas)

Це один із найкрутіших візуальних способів побудови макета. Ви можете буквально "намалювати" структуру в CSS.

```
.layout {
  display: grid;
  grid-template-areas:
    "header header header"
    "sidebar main main"
    "footer footer footer";
}

.header {
  grid-area: header;
```

```
}
.sidebar {
  grid-area: sidebar;
}
.main {
  grid-area: main;
}
.footer {
  grid-area: footer;
}
```

Тепер, для адаптивного дизайну, вам достатньо у медіа-запиті просто змінити "малюнок" у властивості `grid-template-areas`.

6. Адаптивний дизайн: медіа-запити (Media Queries)

Адаптивний веб-дизайн (RWD) означає, що ваш сайт динамічно підлаштовується під розмір екрану користувача, орієнтацію пристрою та інші фактори.

Основним інструментом RWD є **Медіа-запити**. Вони дозволяють застосувати блок CSS-коду лише тоді, коли браузер відповідає певній умові (наприклад, ширина екрану менше `768px`).

```
/* Базові стилі */
.content {
  font-size: 16px;
  grid-template-columns: 1fr 1fr;
}

/* Правило спрацює лише на екранах ширших або рівних 768px */
@media screen and (min-width: 768px) {
  .content {
    font-size: 18px; /* Збільшуємо шрифт на великих екранах */
  }
}

/* Правило спрацює лише на екранах до 480px включно (телефони) */
@media screen and (max-width: 480px) {
  .content {
    grid-template-columns: 1fr; /* Робимо 1 колонку замість двох */
  }
}
```

Головне правило адаптивності: **Обов'язково** додавайте мета-тег Viewport всередину `<head>`, інакше медіа-запити на телефонах не працюватимуть коректно: `<meta name="viewport" content="width=device-width, initial-scale=1.0">`

7. Методології адаптивного дизайну: Mobile First vs Desktop First

Існує дві принципові стратегії написання адаптивного CSS:

1. Desktop First (спочатку десктоп)

- **Підхід:** Ви спочатку пишете CSS для великих моніторів.
- **Як працює:** Використовуються медіа-запити `@media (max-width: ...)` для поступового звуження та спрощення макету до мобільної версії.
- **Плюси:** Зручно, якщо основна аудиторія сайту — це користувачі ПК (B2B портали, складні адмін-панелі).

2. Mobile First (спочатку мобільні)

- **Підхід:** Ви пишете базовий CSS під найменші екрани перш за все (відповідно, базовий дизайн є лінійним).
- **Як працює:** Використовуються медіа-запити `@media (min-width: ...)` для поступового розширення та ускладнення макету з ростом екрану.
- **Плюси:** Є сучасним стандартом (оскільки 60-70% трафіку генерують мобільні пристрої). Це також покращує швидкодію на телефонах, бо браузеру не доводиться читати складні "десктопні" правила лише для того, щоб переписати їх мобільними. Пошуковики (Google) також індексують сайти за Mobile First принципом.

Висновки

1. CSS Grid — найкращий інструмент для створення двовимірних структурних макетів сторінки. Він оперує поєднанням рядків та колонок.
2. Фракція (`fr`) є потужним засобом розподілу доступного простору в Grid, який значно спрощує розрахунки.
3. Комбінація функцій `repeat()`, `auto-fit` та `minmax()` здатна створювати надзвичайно потужні та гнучкі адаптивні сітки навіть без медіа-запитів.
4. Властивість `grid-template-areas` робить CSS код семантичним та зрозумілим.
5. Для повноцінної адаптивності використовуються Медіа-запити (`@media`), які дозволяють підміняти стилі залежно від умов (зокрема, ширини вікна).
6. Підхід **Mobile First** є поточним галузевим стандартом при розробці адаптивних веб-додатків.

Джерела

1. A Complete Guide to CSS Grid (CSS-Tricks) - найкращий візуальний гайд по Grid.
2. MDN Web Docs: CSS Grid Layout
3. Grid Garden (https://cssgridgarden.com/) - інтерактивна гра для вивчення Grid.
4. Google Web Fundamentals: Responsive Web Design Basics

Запитання для самоперевірки

1. У яких ситуаціях варто використовувати Flexbox, а у яких — CSS Grid?
2. Що таке лінія сітки (Grid Line) і з якої цифри починається їх нумерація?
3. Як працює одиниця вимірювання `fr` (Fraction)? Наведіть приклад.
4. Поясніть принципову різницю між `auto-fill` та `auto-fit`.
5. Як властивість `span` впливає на розташування елемента по сітці?
6. Що таке іменовані області (`grid-template-areas`) і як з ними працювати? Наведіть приклад синтаксу для 2-колонкового макета з `header` і `footer`.
7. Який мета-тег є обов'язковим для коректної роботи адаптивного дизайну на мобільних пристроях?
8. Поясніть концепцію Mobile First. Які медіа-запити (`min-width` чи `max-width`) використовуються у цьому підході?